

CSE 4621: Machine Learning

Linear Regression and

Regularization

Lecture 4

Ishmam Tashdeed
Lecturer, CSE IUT

Lecture Outline

- Multivariate Linear Regression
 - GD on multiple features
 - Polynomial regression
 - Normal Equation
- Regularization
 - Bias-variance trade-off
 - The curse of dimensionality
 - L2 and L1 regularization
 - Model selection

What is Multivariate Linear Regression?

Suppose you want to buy a house in Dhaka and the only thing that mattered for the price was the size of the house.

Size (sq. feet)	Price (\$)
1506	1030
1640	2300
...	...
852	581

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

What is Multivariate Linear Regression?

But, the price of a house depends on a multitude of factors:

Size (sq. feet)	Number of Bedrooms	Number of Floors	Age of the house (years)	Price (\$)
1506	3	3	12	1030
1640	4	2	5	2300
...	...	...	...	...
852	1	1	23	581

What is Multivariate Linear Regression?

Given the input features x , the goal is to learn $h : x \rightarrow y$ where f is a **weighted sum** of different values (as h is supposed to be a linear function)

Notation:

m = Number of training examples

n = Number of features

$x_j^{(i)}$ = j^{th} feature value of the i^{th} training example

$(x^{(i)}, y^{(i)})$ = i^{th} training example



**Multivariate
Regression**



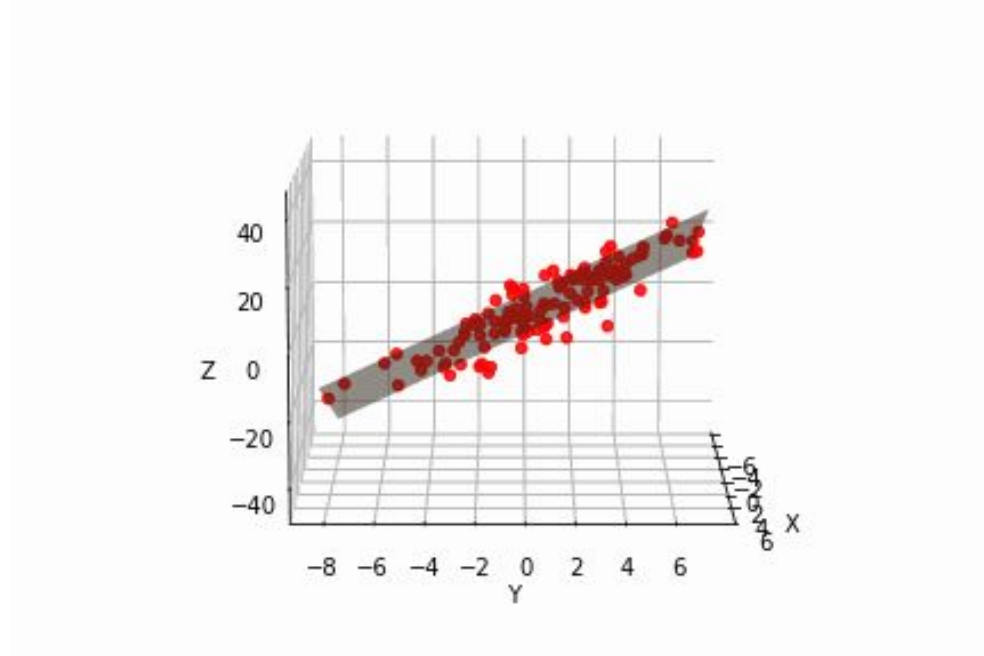
**Machine Learning
in high-dimensional
feature spaces**

What is Multivariate Linear Regression?

How would we represent each training example $x^{(i)}$?

Size (sq. feet) (x_1)	Number of Bedrooms (x_2)	Number of Floors (x_3)	Age of the house (years) (x_4)	Price (\$) (y)
1506	3	3	12	1030
1640	4	2	5	2300
...	...	...	...	...
852	1	1	23	581

What is Multivariate Linear Regression?



What is Multivariate Linear Regression?

Training Data: $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^m$

Hypothesis function: $h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$

For convenience, $x_0 = 1$

$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^{n+1} \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \theta_3 \\ \vdots \\ \theta_n \end{bmatrix} \in \mathbb{R}^{n+1}$$

$$h_{\theta}(x) = [\theta_0, \theta_1, \dots, \theta_n] \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{bmatrix}$$
$$h_{\theta}(x) = \theta^T x$$

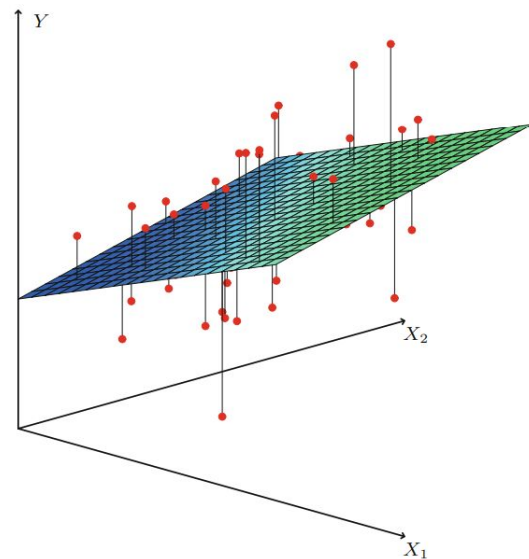
Cost Function

Residual for the i^{th} example: $e^{(i)} = y^{(i)} - \hat{y}^{(i)} = y^{(i)} - (\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n)$

Mean Squared Error (MSE) loss: $J(\theta) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$

The $(1/2)$ is a convention that simplifies the gradient descent process

The goal for any cost function is to minimize the loss/cost for all θ



Gradient Descent

Previously,

repeat until convergence {

$$\theta_0 = \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_1 = \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

}

The new equation is a generalized version of the previous equation

Now,

repeat until convergence {

$$\theta_j = \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)}$$

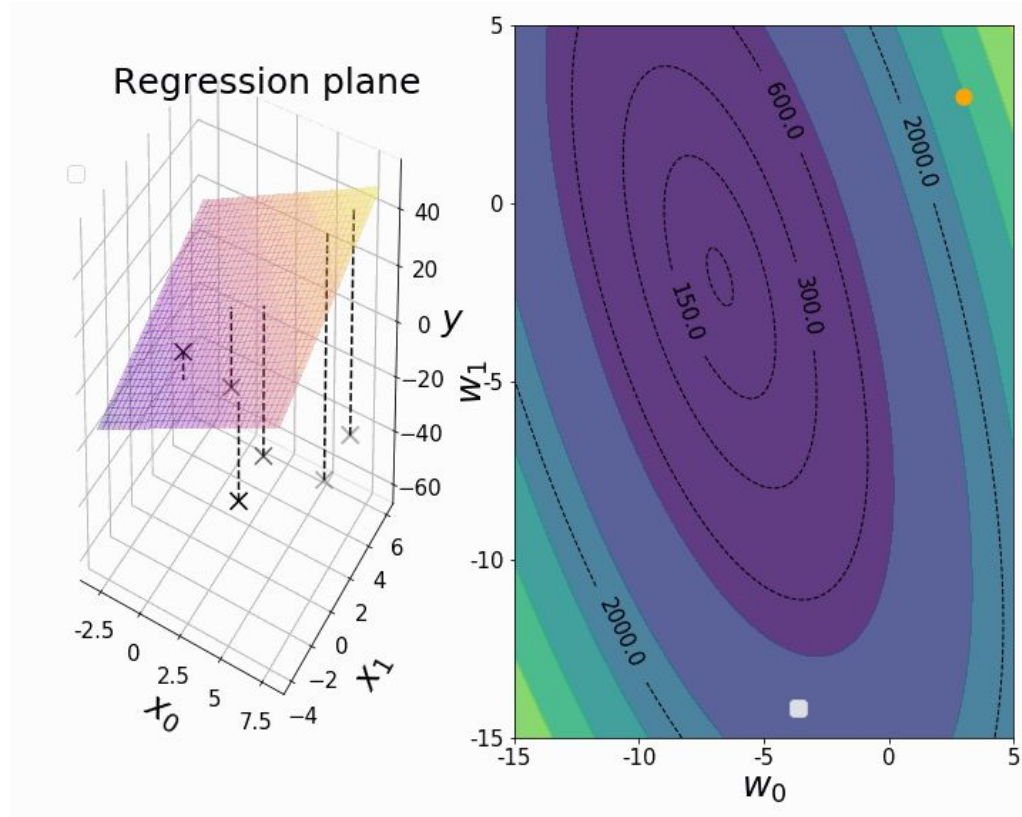
}

$$\theta_0 = \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_0^{(i)}$$

$$\theta_1 = \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_1^{(i)}$$

$$\theta_n = \theta_n - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_n^{(i)}$$

Gradient Descent

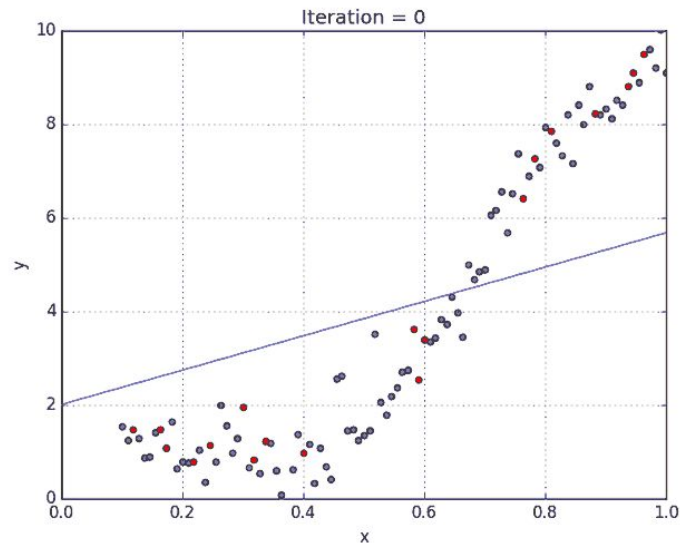


Polynomial Regression

What if your data is not linear?

No matter how many steps of linear regression you try, your line will never be the “best fit line”

We need to update our hypothesis function!



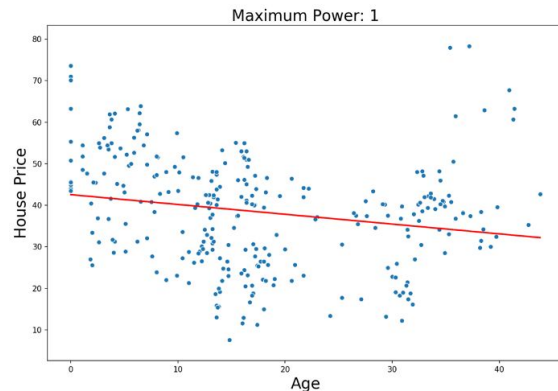
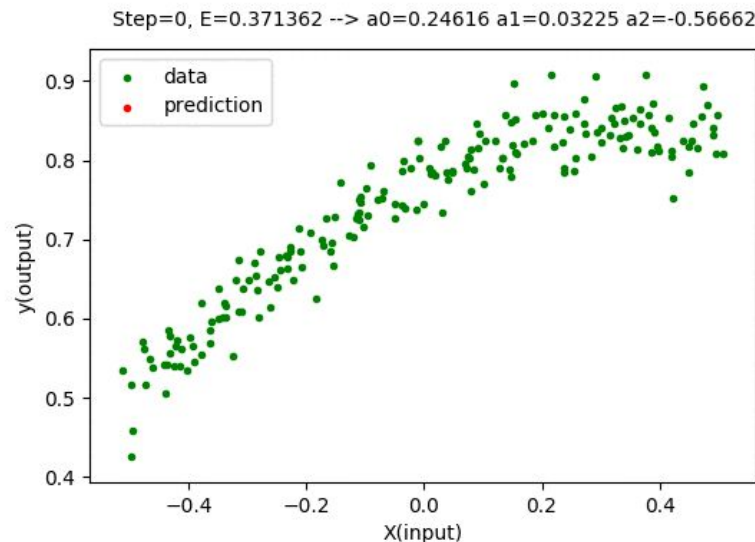
Polynomial Regression

$$h_{\theta}(x) = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2^2$$

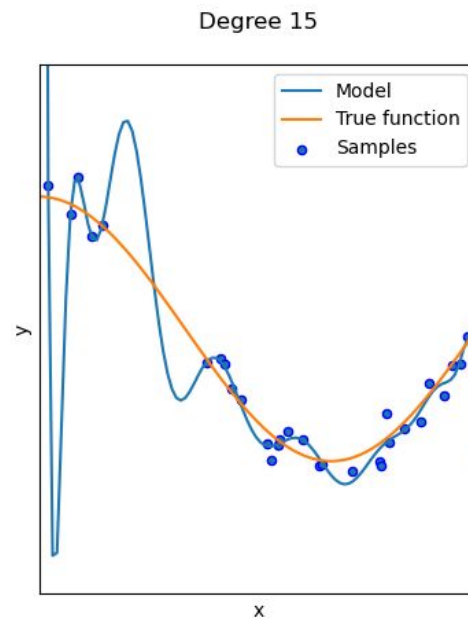
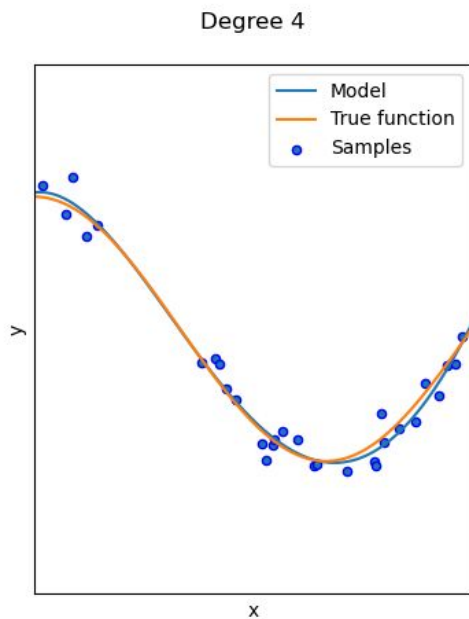
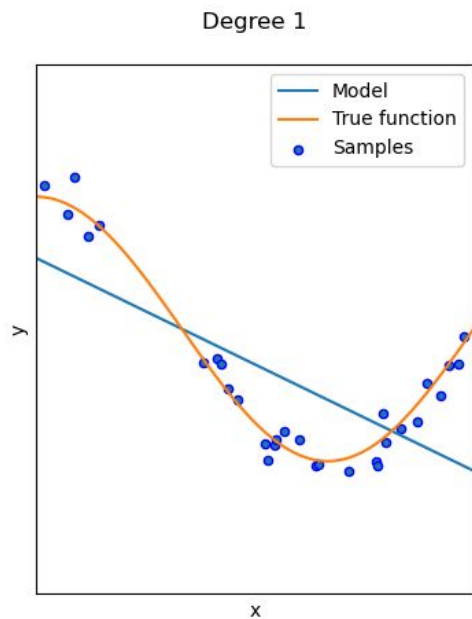
$$h_{\theta}(x) = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2^2 + \theta_3 x_3^3$$

The more parameters we add to the function, the better it can accommodate the data!

Why don't we keep increasing the number of parameters?



Polynomial Regression

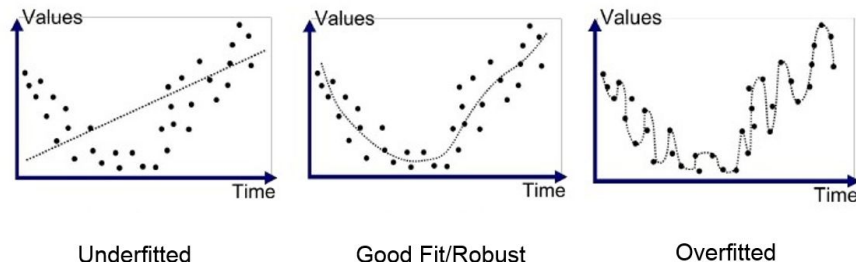


Overfitting Problem

If we have too many features, the learned hypothesis function may fit the training set very well ($J(\theta) \approx 0$), but fail to generalize to newer / unseen examples

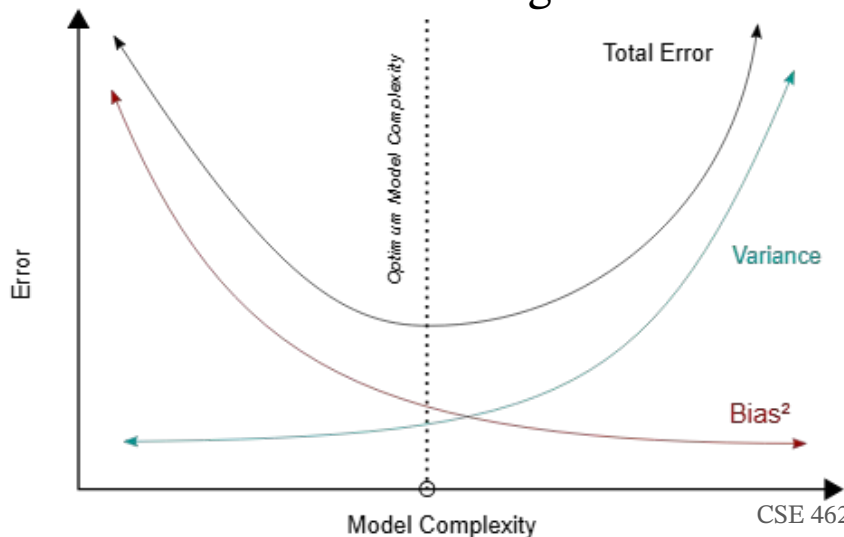
Three possible regimes:

- Underfit: Model is too simple to capture the pattern in the data → High Bias, High Variance
- Good fit: Model generalizes well → Low Bias, Low Variance
- Overfit: Model memorises noisy training data → Low Bias, High Variance



Bias-Variance Trade-off

- One of the core challenges of supervised machine learning
- Involves managing *bias* (error from a simplistic model, leading to underfitting) and *variance* (error from sensitivity to noisy training data, leading to overfitting)
- Need to find a balance to ensure the model generalizes well to new, unseen data



Curse of Dimensionality

As the number of features (dimensions) increases, the volume of the space grows exponentially, making the same amount of data points appear extremely sparse.

Coined by Richard E. Bellman, while talking about the computational complexity of dynamic programming.

- To maintain the same "density" or statistical reliability as in low dimensions, you would need an exponentially larger dataset.
- In high-dimensional spaces, traditional distance measures (like Euclidean distance) lose their effectiveness because the difference between the closest and farthest points tends to zero.

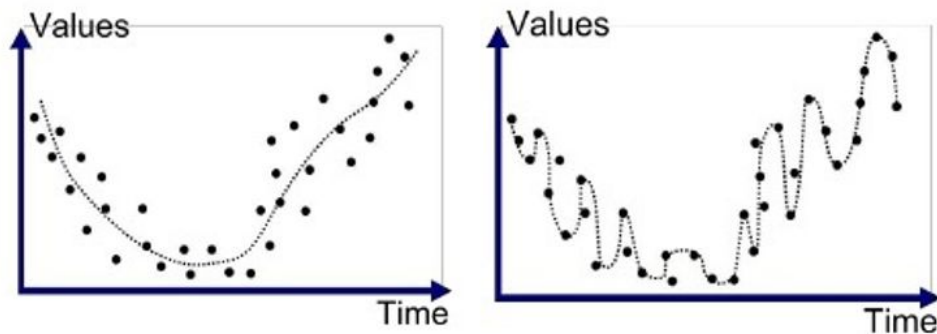
Overfitting Problem

How to get rid of overfitting?

- Reduce the number of features
 - Manually select which features to use
 - Some algorithm selects which features to use
- Regularization
 - Keep all the features, but somehow reduce their influence on the parameters
 - Works well when there are a lot of features but all of them contribute to the prediction



Regularization



$$h_{\theta}(x) = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2^2 + \theta_3 x_3^3 + \dots + \theta_n x_n^n$$

If we could decrease the values of $\theta_3, \theta_4, \theta_5, \dots, \theta_n$, their influence on $h_{\theta}(x)$ would be less. How can we penalize the optimization function (gradient descent) so that these values are punished?

Regularization

Small values for parameters $\theta_1, \theta_2, \theta_3, \dots, \theta_n$:

- Hypothesis function is less sensitive on input data
- Simpler hypothesis function
- Less prone to overfitting

Higher values for parameters $\theta_1, \theta_2, \theta_3, \dots, \theta_n$:

- Much more sensitive to slight changes in input data
- Complex (wiggly) hypothesis function
- High variance, thus prone to overfitting

Regularization

General regularized loss: $J_{reg}(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \Omega(\theta) \right]$

Here, $\lambda \geq 0$ is the regularization hyperparameter

- If $\lambda = 0$, no regularization $\rightarrow$ Prone to overfitting
- If $\lambda \approx \infty$, max regularization, all $\theta \approx 0 \rightarrow$ Prone to underfitting
- Optimal $\lambda^* \rightarrow$ Best for generalization

What $\Omega(\theta)$ should we choose?

Ridge Regularization

Ridge regularization (L2 regularization) cost function:

$$J_{ridge}(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

Thus, the gradient is different for θ_0 compared to the other parameters:

$$\frac{\partial}{\partial \theta_0} J_{ridge}(\theta) = \frac{1}{m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_0^{(i)} \right] \quad \text{For } j = 0, \text{ and } x_j = 1$$
$$\frac{\partial}{\partial \theta_j} J_{ridge}(\theta) = \frac{1}{m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} \theta_j \right] \quad \text{For } j \geq 1$$

Ridge Regularization

repeat until convergence {

$$\theta_0 = \theta_0 - \alpha \frac{1}{m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_0^{(i)} \right]$$

$$\theta_j = \theta_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} \theta_j \right]$$

}

Ridge Regularization

The effect of ridge regularization is prominent if we reorganize the equation:

$$\theta_j = \theta_j(1 - \alpha \frac{\lambda}{m}) - \alpha \left[\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} \right]$$

- First decreases the weight θ_j by some amount
- Then updates the decreased weight using the gradient

Ridge penalises large θ_j equally, so features on different scales get unfair treatment. It is critical to standardise the features before ridge regression is applied!

THEY SAID I COULD BE ANYTHING

OVERFIT TODAY,
REGULARIZE TOMORROW

$$J_{\text{ridge}}(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

aka
the chill pill

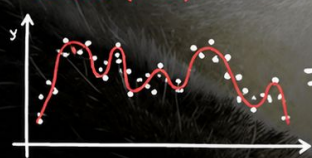
LESS OVERFITTING ✓

MORE GENERALIZATION ✓

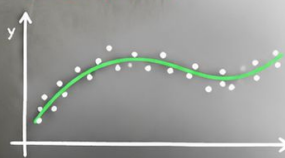
SMALLER EGOS (WEIGHTS) ✓

BETTER SLEEP ✓

BEFORE ($\lambda = 0$)



AFTER ($\lambda > 0$)



Ridge: Because **real models**
don't chase every noise.



WARNING:

Too much λ and you
become linear. Sad.

Lasso Regularization

Lasso regularization (L1 regularization) cost function:

$$J_{lasso}(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n |\theta_j| \right]$$

Thus, the gradient is different for θ_0 compared to the other parameters:

$$\frac{\partial}{\partial \theta_0} J_{lasso}(\theta) = \frac{1}{m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_0^{(i)} \right] \text{ For } j = 0, \text{ and } x_j = 1$$

$$\frac{\partial}{\partial \theta_j} J_{lasso}(\theta) = \frac{1}{m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} \theta_j \right], \text{ if } \theta_j > 0$$
$$\frac{\partial}{\partial \theta_j} J_{lasso}(\theta) = \frac{1}{m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} - \frac{\lambda}{m} \theta_j \right], \text{ if } \theta_j < 0$$

For $j \geq 1$

Lasso Regularization

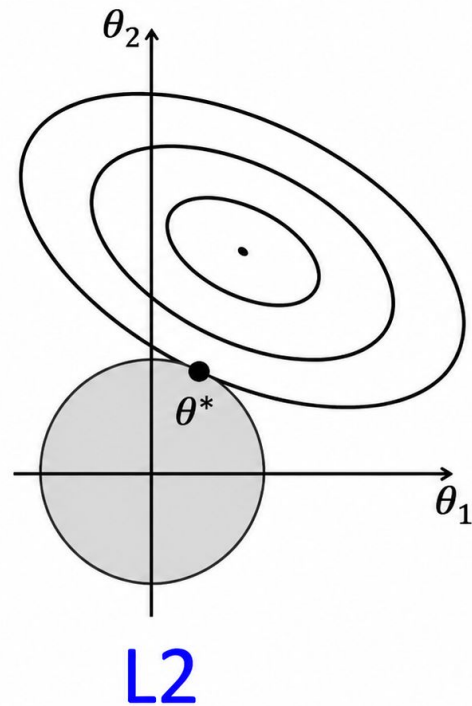
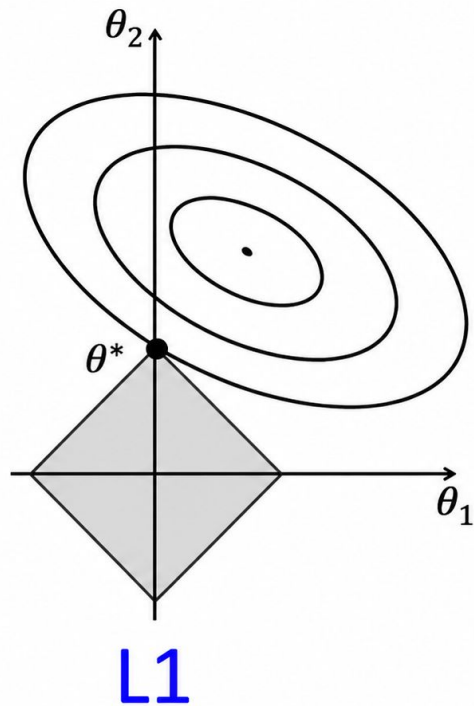
repeat until convergence {

$$\theta_0 = \theta_0 - \alpha \frac{1}{m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_0^{(i)} \right]$$

$$\theta_j = \theta_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} \pm \frac{\lambda}{m} \theta_j \right]$$

}

Regularization



Regularization

In L1 regularization,

- Unimportant feature parameters shrink to zero → Sparse solution
- Can be used for feature selection → Automatically set unimportant parameters to zero

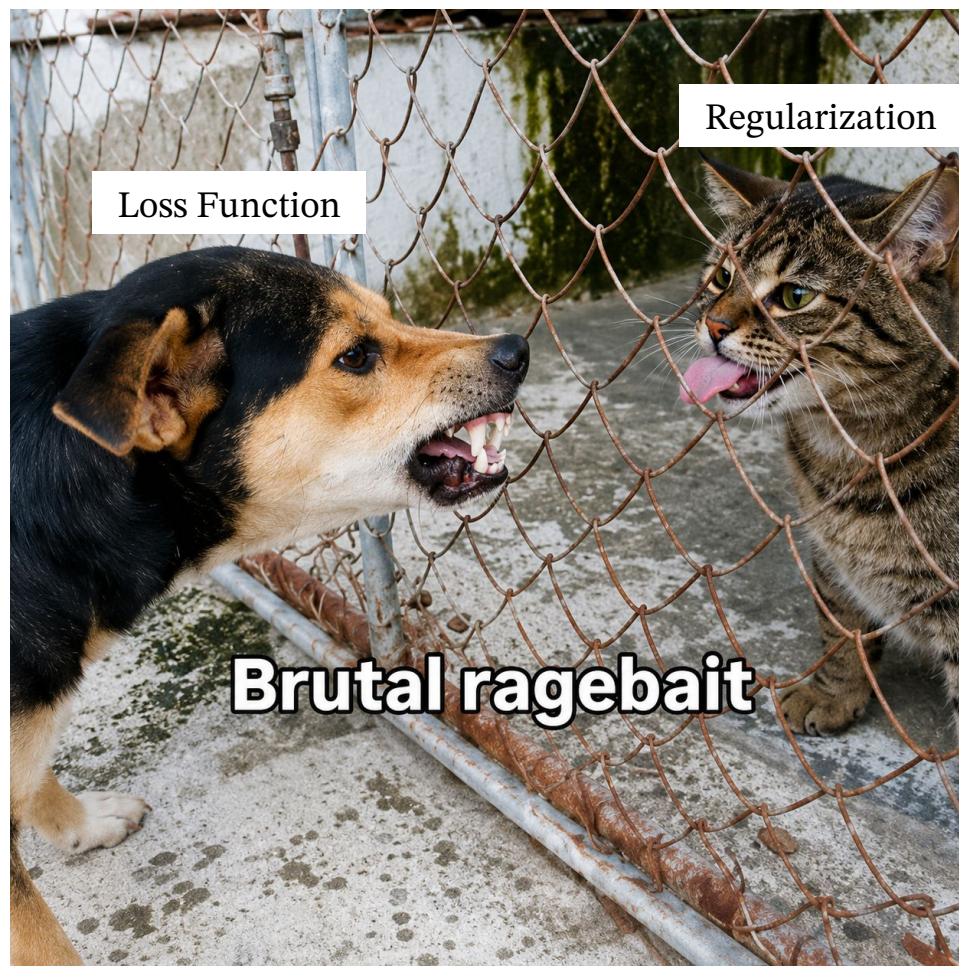
In L2 regularization,

- Forces the parameters to be small but does not make them zero → Dense solution
- Less likely to produce zero weight coefficients
- Performs better when all parameters influence the output

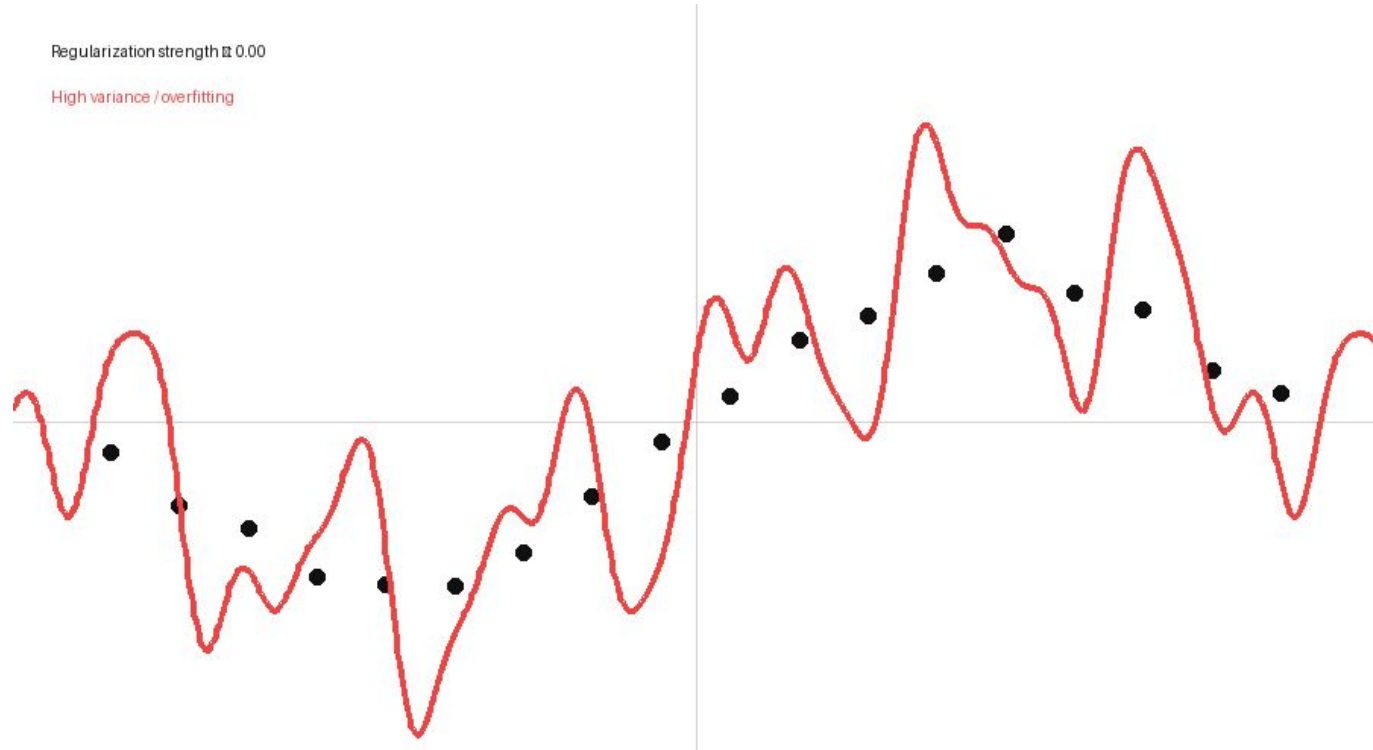
Regularization

Some other types of regularization methods:

- Elastic Net
- Dropout
- Early Stopping



Regularization



Textbook Chapters

- [Goodfellow] - Chapter 7
- [Bishop] - Chapter 3.1.4
- [Alpaydin] - Chapter 4.7

Recommended Resources

1. [Linear Regression - Clearly Explained](#)
2. [Desmos - Playing with Polynomials](#)
3. [The science behind why we see faces in nature](#)
4. [The Math Behind “The Curse of Dimensionality”](#)
5. [Do you know overfitting and underfitting?](#)
6. [Curse of Dimensionality Comic](#)

Thank you